

Identidade visual e design system

Toda a aparência do app fica centralizada em **um único arquivo CSS**, `src/index.css`, usando variáveis (custom properties) — não há Tailwind nem CSS-in-JS. Isso significa que ajustar um token no topo do arquivo propaga a mudança pro sistema inteiro (sidebar, cards, tabelas, botões, modais), sem precisar tocar em cada página.

Tema claro/escuro

Controlado por `src/contexts/ThemeContext.tsx`:

- Estado `'dark' | 'light'`, persistido em `localStorage` (chave `km-theme`), padrão `'dark'`.
- Aplica o tema escrevendo `data-theme` na tag `<html>` (`document.documentElement`).
- `src/index.css` define os tokens duas vezes: uma vez em `:root` (tema escuro, padrão) e de novo em `:root[data-theme='light']` (sobrescreve só o que muda).
- Hook de acesso: `useTheme()` (`src/hooks/useTheme.ts`) → `{ theme, toggleTheme, setTheme }`. Botão de troca fica no header (`AppLayout.tsx`).

Status atual: o tema escuro ("Black Mode") já passou por uma especificação de design completa (paleta fornecida pelo usuário + refatoração de profundidade/sombras/gradientes inspirada em um dashboard financeiro de referência). O tema claro ainda é um placeholder funcional — mantém o app usável em modo claro, mas ainda não recebeu uma especificação de design própria ("White Mode"). Ao adicioná-la, o padrão é: editar só o bloco `:root[data-theme='light']` em `src/index.css`, seguindo a mesma lista de tokens do tema escuro.

Tokens de cor (tema escuro)

Definidos em `:root` no topo de `src/index.css`, em três grupos:

Marca (`--km-*`) — não mudam entre os temas

Token	Uso
<code>--km-cyan</code>	Cor de detalhe/destaque principal (ciano)
<code>--km-teal</code>	Detalhe secundário / hover do accent
<code>--km-blue-strong</code>	Azul forte — gráficos, progress
<code>--km-periwinkle</code> , <code>--km-gray-blue</code> , <code>--km-lavender</code> , <code>--km-navy-deep</code>	Paleta de apoio para categorias adicionais em gráficos
<code>--km-critical</code>	Vermelho reservado para status "quebrado"/crítico

Acentos "premium" (cross-theme)

Token	Uso
<code>--accent-primary</code>	Gradiente azul→violeta — fundo dos botões primários e do CTA principal
<code>--accent-secondary</code>	Índigo — estado ativo da navegação
<code>--accent-cyan</code>	Alias de <code>--km-cyan</code>
<code>--accent-purple</code>	Violeta — detalhe/gráficos

Superfícies e "casco" da interface (mudam por tema)

Token	Nível de profundidade	Uso
<code>--bg</code> / <code>--bg-app</code>	0	Fundo geral da página
<code>--surface-1</code>	2	Containers e cards padrão (stat card, chart card, tabela)
<code>--surface-2</code>	2	Cards "elevados" (donuts, gráfico técnico, modais)
<code>--surface-3</code>	3	Subcards/tiles (KPIs de telefonia)
<code>--surface-hover</code> / <code>--surface-active</code>	interação	Hover neutro / estado selecionado (índigo translúcido)
<code>--shell-bg</code> / <code>--bg-sidebar</code> / <code>--bg-header</code>	1	Sidebar e header — sempre um tom "fora" do conteúdo
<code>--border-default</code> / <code>--border-highlight</code>	—	Borda padrão / borda que acende no hover e no foco
<code>--text-h</code> / <code>--text-primary</code>	—	Texto de destaque (branco)
<code>--text</code> / <code>--text-muted</code> / <code>--text-secondary</code>	—	Texto secundário
<code>--shadow-card</code> / <code>--shadow-elevated</code> / <code>--shadow-glow</code>	—	Sombra de card / sombra de modal (mais profunda) / anel de foco
<code>--canvas-glow-center/mid/edge</code>	—	Brilho radial de fundo do Dashboard (tamanho fixo em px, não em %, pra não "esticar" em páginas compridas)
<code>--hero-grad-top/base</code>	—	Gradiente vertical dos cards de KPI do topo do Dashboard
<code>--radius-sm/md/lg/pill</code>	—	9px (inputs/botões) / 16px (cards) / 20px (containers) / 999px (badges/chips)

`--option-bg` é a cor **sólida** do fundo das opções de qualquer `<select>` aberto (`#0f2740` no escuro, `#ffffff` no claro). Ela existe porque `--surface` é gradiente no tema escuro, e o Chrome/Windows desenha a lista aberta de um select usando o fundo do próprio select: com fundo semitransparente ou gradiente, a lista virava branca e o texto branco sumia. A regra global `select option, select optgroup` em `index.css` usa esse token e vale para todos os selects do app.

Regra importante ao adicionar um token novo: se ele for usado fora do bloco `:root` "padrão", **também precisa de um valor equivalente no bloco** `[data-theme='light']` — senão o tema claro herda o valor do tema escuro sem querer (já aconteceu nesse projeto e foi corrigido mais de uma vez durante o desenvolvimento).

Paleta de dados (gráficos)

Usada nas constantes `BLUE` / `TEAL` / `CRITICAL` / `WARNING` / `MUTED` em `DashboardPage.tsx` e `CARRIER_COLOR_BY_NAME` em `TelephonyDashboardPanel.tsx` (não são CSS — Recharts precisa de cores literais nos componentes `Bar` / `Line` / `Area` / `Cell`).

Cores das operadoras (telefonia do Dashboard — gráfico de custo, rosca de contagem e tiles de custo), fixas pelo nome da operadora e bem separadas no círculo cromático para as linhas do gráfico não se confundirem:

Operadora	Cor
Claro	<code>#ef4444</code> (vermelho)
Oi	<code>#22c55e</code> (verde)
TIM	<code>#3b82f6</code> (azul)
Vivo	<code>#a855f7</code> (roxo)
Operadora nova sem cor definida	<code>FALLBACK_PALETTE</code> (âmbar, ciano, rosa, lima), em ordem alfabética

Regras seguidas (ver skill de dataviz usada no projeto):

- Cor por identidade fixa, nunca "a n-ésima cor de uma lista cíclica".
- Texto nunca herda a cor da série — um número sempre fica na cor de texto neutra; a identidade da série é só o ponto/traço/badge ao lado.
- Paleta validada com um script de acessibilidade (contraste, separação por daltonismo) antes de aplicada — a única cor que ficou abaixo do ideal foi o "cinza-azulado" usado pra categorias neutras/"outros", mantido porque sempre aparece com rótulo direto ao lado (nunca só a cor).

Marca "KM"

- **Arte neon** (<src/assets/branding/km-logo.png>): arte gerada (neon, detalhada) fornecida pelo usuário, com o fundo preto original removido (canal alfa gerado a partir do brilho de cada pixel — as áreas antes pretas ficam transparentes, o próprio glow permanece

intacto). Usada em:

- Logo da sidebar **no tema escuro** (`.app-sidebar-logo-img` , `object-fit: contain`).
- Banner no topo do card de login **nos dois temas** (`.auth-logo-hero`). No tema claro recebe um `filter: drop-shadow( ... ) saturate(1.25)` sutil, porque o glow ciano sozinho fica lavado sobre fundo branco.
- **Arte do tema claro** (`src/assets/branding/km-logo-light.png`): versão metálica (contorno prateado, miolo azul-marinho, circuitos ciano), PNG com transparência real. Usada como logo da sidebar **no tema claro**. A troca é feita em `AppLayout.tsx` pelo `theme` do `useTheme()` .
 - Atenção ao receber uma arte nova: várias ferramentas de IA entregam PNG com o **xadrez de "transparência" desenhado na imagem** (formato RGB, sem canal alfa). Antes de usar, confirme que o arquivo é RGBA e tem pixels realmente transparentes.
- **Favicon** (`public/favicon.svg`): versão compacta e estática do mesmo monograma, com variante clara/escura via `@media (prefers-color-scheme)` (independente do toggle de tema do app — segue o SO/navegador).

Arquivos da identidade visual anterior (BIOND) ainda existem no repositório mas não são mais referenciados por nenhum componente: `src/assets/biond-logo.png` e a pasta `Imagens/Logos/` . Ficam guardados até uma decisão de removê-los de vez.

Ícones

Biblioteca: `lucide-react` — usada na sidebar (um ícone por item de navegação), nos KPIs do Dashboard (Laptop/Mouse/Smartphone) e nos botões de tema/logout do header. Não há emojis usados como ícone em nenhum lugar do app (trocados deliberadamente durante o redesign).

Microinterações

Transições de 150–220ms em: hover de card (`translateY(-2px)` + borda que acende), hover de botão (aumento de brilho via `filter: brightness()` , não troca de cor), foco de input/select (anel de destaque via `box-shadow` , nunca só troca de cor — importante para acessibilidade), item de menu ativo. Deliberadamente **sem** animações contínuas nem efeitos chamativos (glow excessivo, blur pesado) — ver o comentário de performance em `index.css` antes de adicionar `backdrop-filter` em qualquer lista/repetição de elementos (hoje só o header do app usa blur, e só ali).